WEEK 1 · LESSON 5 OF 24 · HANDOUT

k-Nearest Neighbours: Learning from Similar Cases

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=HVXime0nQeI — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyData: Know Thy Neighbor — an introduction to scikit-learn and k-NN — https://www.youtube.com/watch?v=e3-DjsKhFug

🎓 **Go deeper:** Data School: Getting started in scikit-learn with the famous iris dataset — https://www.youtube.com/watch?v=hd1W4CyPX58

Your goal in this lesson: Explain how k-nearest neighbours classifies a new case, show why scaling matters for it, and choose a good value of k.

The idea: ask the neighbours

k-nearest neighbours (**k-NN**) is the simplest model to understand. To classify a new borrower:

1. Measure the **distance** from the new borrower to every borrower in the training data.
2. Take the **k** closest ones (for example k = 5).
3. **Vote**: if 4 of the 5 neighbours defaulted, predict "default".

There is no real training step — k-NN just **remembers** the training data. That makes it a **lazy learner**: fast to train, slow to predict on big data.

Distance

For two borrowers with features (a_1, a_2) and (b_1, b_2), the straight-line (Euclidean) distance is:

> $$distance = \sqrt{((a_1 - b_1)^2 + (a_2 - b_2)^2)}$$

— the same Pythagoras you used for vector length in Math & Python for AI, with as many features as you have.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

Why scaling is essential here

Income is measured in hundreds of thousands of RWF; late payments are 0, 1 or 2. Without scaling, a difference of 1,000 RWF in income counts more than a difference of 2 late payments — nonsense. Let's measure it:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

scaler = StandardScaler().fit(X_train)
X_train_s, X_test_s = scaler.transform(X_train), scaler.transform(X_test)

raw = KNeighborsClassifier(n_neighbors=15).fit(X_train, y_train)
scaled = KNeighborsClassifier(n_neighbors=15).fit(X_train_s, y_train)
print("Unscaled:", accuracy_score(y_test, raw.predict(X_test)))
print("Scaled:  ", accuracy_score(y_test, scaled.predict(X_test_s)))
```

Scaling lifts accuracy from about **0.82** to about **0.84**. On data where the units differ even more, the gap can be huge.

Choosing k

```
for k in [1, 5, 15, 25, 51]:
    knn = KNeighborsClassifier(n_neighbors=k).fit(X_train_s, y_train)
    print(k, accuracy_score(y_test, knn.predict(X_test_s)))
```

- **k = 1** copies the single closest borrower — it follows noise and scores worst (about **0.77**). This is **overfitting**.
- **Large k** smooths too much: with k = 1,500 every prediction becomes the majority class. This is **underfitting**.
- A middle value (here around 15–25) works best.

Choosing k by looking at the **test set** is actually cheating a little — you are tuning to the exam. In Week 3 you will choose k properly with **cross-validation**.

Tip: for two classes, use an **odd k** so votes cannot tie.

k-NN for numbers

`KNeighborsRegressor` predicts a number by **averaging** the neighbours' values — for example the average rent of the 10 most similar homes. Estate agents do this in their heads: "similar flats nearby rent for about…".

Strengths and weaknesses

Strengths	Weaknesses
Very easy to explain	Slow to predict on large data
No training time	Needs scaling
Works for any shape of boundary	Struggles with many features (distances lose meaning)

Key terms

- **k-nearest neighbours** — predict from the k most similar training examples.
- **Euclidean distance** — straight-line distance between two examples.
- **Lazy learner** — stores the data instead of learning a formula.
- **Overfitting** — following noise in the training data (k too small).
- **Underfitting** — too simple to capture the pattern (k too large).

Remember this: k-NN predicts from similar past cases; it only works well when features are scaled and k is neither too small nor too large.

Name: _____ Date: _____

Activity

Task 1

Plot test accuracy for k from 1 to 60 (`plt.plot`). Where is the best region?

Task 2

Use `KNeighborsRegressor(n_neighbors=10)` on the homes data (one-hot district, scaled numbers). Compare its MAE with linear regression from Lesson 3.

Task 3

Try `weights="distance"` so closer neighbours get a bigger vote. Does it help?

Task 4

Explain why k-NN would be slow for a lender with 10 million past loans.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. How does k-NN classify a new example?
   - A. By the first row of the data
   - B. By random choice
   - C. By a vote of the k most similar training examples
   - D. By a straight-line formula
2. Why must features be scaled for k-NN?
   - A. So features with big numbers do not dominate the distance
   - B. To save memory
   - C. Because k must be odd
   - D. It is not needed
3. What usually happens with k = 1?
   - A. An error
   - B. Underfitting
   - C. Overfitting: the model follows noise
   - D. Perfect test accuracy
4. Why is k-NN called a lazy learner?
   - A. It is slow to train
   - B. It never predicts
   - C. It uses few features
   - D. It stores the training data instead of learning a formula
5. For a yes/no problem, why choose an odd k?
   - A. It is faster
   - B. So votes cannot tie
   - C. It improves scaling
   - D. Even numbers are not allowed
6. Which model predicts a number from the average of similar cases?
   - A. LogisticRegression
   - B. KMeans
   - C. DummyClassifier
   - D. KNeighborsRegressor

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.